

From web game to **App Store and Play Store**, on our own VPS

The plain-words version of the launch plan for the team. Canonical source: LAUNCH_PLAN.md in the repo.

~33 dev days 8 to 10 weeks to live 16 decisions waiting on you 9 phases

The whole plan in 13 lines

- 1** **Wrap, don't rewrite.** The React app ships inside a Capacitor shell. One codebase for web, iOS and Android.
- 2** **Backend leaves Supabase cloud and runs on our VPS.** We run the open-source server software ourselves (Postgres, auth, API, realtime). No Supabase account, no bill, no supabase.co anywhere. Only the URL changes in the app. Web and apps share one database. See the "Off Supabase" section.
- 3** **That same move fixes the Google login text.** The consent screen shows the callback domain. Ours becomes `api.letterlock.raltech.dev`, then we verify the brand so it reads "Letterlock".
- 4** **Ads = Google AdMob.** Interstitial only between games, rewarded ads for an extra skip or hint, banner only on menus. Never during a question.
- 5** **Remove Ads = one purchase, about \$3.99,** through Apple and Google billing via RevenueCat, mirrored to a flag on the account so it follows the login, not just the phone.
- 6** **Purchases belong to the Apple ID or Google account by default.** Our server flag is what makes them follow the user across iPhone, Android and web. The "Remove Ads" tab explains it.
- 7** **The song, TV-clip and iTunes-melody packs cannot ship in the store build.** Apple's guideline 5.2.5 forbids using iTunes previews "as the soundtrack to a game". Replace with self-made public-domain melodies and text clues.
- 8** **Store requirements we do not have yet:** in-app account deletion, privacy policy and terms pages, Sign in with Apple (mandatory because we offer Google login), real PNG icons, offline handling.

- 9 **"AI-looking code" is a myth.** Neither store checks who wrote the code. They reject thin web wrappers, crashes, placeholders and missing privacy features. The "Store review" tab is the real checklist.
- 10 **Paperwork first:** Apple Developer (\$99/yr), Google Play (\$25 once), AdMob, RevenueCat, and a differentiated store name because exact "Letterlock" is already taken.
- 11 **Ops we must add:** backups, uptime monitoring, crash reporting, a version gate, `app-ads.txt`.
- 12 **Realistic money:** ads earn tens of dollars a month at hundreds of daily players. The \$3.99 purchase will likely out-earn ads early. Do it for reach and polish, not quick revenue.
- 13 **Mobile and TV are first-class targets.** A full mobile experience rehaul (portrait and landscape, no browser bar) built with the frontend-stack pipeline before submission, plus an Android TV / Google TV build controlled with the remote. See the "Mobile & TV" section.

Where we actually are today

AREA	REALITY	CONSEQUENCE
Web hosting	Cloudflare Pages, not the VPS. The VPS hosts nothing for Letterlock yet.	The migration builds the VPS side from scratch.
Backend	Supabase cloud project, ref committed in <code>.env.production</code> .	That ref is the random text on the Google consent screen. We leave it.
Backend footprint	Auth (email code via Resend, Google), 9 tables, ~30 database functions, 12 migrations, realtime broadcast + presence, 2 edge functions, no file storage.	Small and portable. Moving it is realistic.
Account deletion	None for users.	Store blocker on both stores.
Privacy policy, terms	None anywhere.	Store blocker, also needed by AdMob and Google brand verification.
Sign in with Apple	Not built.	Required by Apple 4.8 because Google login exists.
Icons	SVG favicon only.	Need 1024/512/192 PNG set and splash.
Join links	Query params only (<code>?room=</code>).	Add <code>/join/CODE</code> : universal links match paths reliably.
Remote media	1,624 iTunes song previews, 296 iTunes TV clips, ~600 CDN logos, random Flickr photos in charades.	Copyright, reliability and content-safety work (Copyright tab).
Monetization SDKs	Zero.	Ads, purchases, crash reporting are all new.

Decisions waiting on you

Recommendation is listed first. Say "go with the recommendations" and the plan proceeds as written.

#	DECISION	RECOMMENDATION	WHY
D1	Store listing name	Letterlock: Trivia Hex Duel (or Letterlock Party Quiz)	Exact "Letterlock" is taken on both stores by word-puzzle apps. Apple names must be unique.
D2	Developer account type	Organization (RAL, free D-U-N-S number)	Shows the company name; Play organization accounts skip the 12-testers-for-14-days rule.
D3	Song / TV / iTunes-melody packs	Remove from the store build. Grow the 23 self-made melodies to 200+, convert songs and TV to text clues.	Apple 5.2.5 and the iTunes API terms forbid game use. Licensed clips can come later.
D4	Ads placement and price	Interstitial between games (max 1 per 3 to 5 min), rewarded for skip/hint, banner on menus. Remove Ads at \$3.99 .	Matches Play "Better Ads" and Apple UX rules. \$2.99 to \$4.99 is the norm.
D5	Static web hosting	Move the site to the VPS too, keep Cloudflare as DNS only	You asked for "fully on our VPS". One hour once the API is there.
D6	Target audience	13+, not directed at children	Kids Category forbids third-party ads. Keep the Kids pack, skip child-directed marketing.
D7	Over-the-air JS updates	Later. Store builds only at launch.	Content waves need a store update (Play hours, Apple 1 to 2 days). Add Capgo when that hurts.
D8	VPS size	8 GB RAM (4 GB hard minimum)	The backend stack idles near 3 to 4 GB. Tell me the current spec.
D9	Charades images	Drop images, show the word only	Random Flickr photos: content-safety and license risk.
D10	Logo packs	Keep: bundle SVGs, trademark disclaimer, drop the Apple logo, removal contact	How every Logo Quiz app survives.
D11	Fandom packs	Keep as text, label "Unofficial fan trivia", rename the Harry Potter and Pokémon packs, never in store metadata	Those two rights-holders are the most aggressive. Facts are not copyrightable.
D12	Web ads	Skip at launch	AdSense rejects thin-text game pages; a rejection complicates later approval.
D13	Web Remove-Ads purchase	Optional, later, via Paddle or Lemon Squeezy	They handle VAT. Never mention web pricing inside the apps.
D15	TV scope at launch	Android TV / Google TV only (Sony, Philips, TCL, Hisense, Xiaomi, Chromecast, Nvidia Shield), same Android project. LG and Samsung later as HTML5 TV apps. Apple TV not feasible.	LG (webOS) and Samsung (Tizen) have no Play Store but accept HTML5 apps, so our web build can be packaged for them later. Apple TV has no web view at all.
D16	Mobile rehaul before submission	Yes , Phase 1b runs before the store builds, through the frontend-stack pipeline (style pick, taste floor, guidelines review, Playwright device-matrix verify).	Store screenshots and first reviews come from the mobile UI. Shipping the current mobile experience and fixing later burns the launch.

#	DECISION	RECOMMENDATION	WHY
D14	What runs on the VPS	Option A: run the open-source Supabase server stack ourselves (no Supabase account, no bill, our domain). ~3 days, zero app rewrite.	Option B is a fully custom backend (own API, own auth, own websockets): 4 to 6 weeks of rewrite for the same result. The database is plain Postgres either way, so B stays possible later. See the "Off Supabase" section.

The nine phases

Phases 1 and 2 run in parallel. Phase 0 starts today because it is mostly waiting.

0 Paperwork 1 to 2 weeks wait	1 Store-ready web 5 days	1b Mobile rehaul 6 days
2 Backend to VPS 3 days + cutover	3 Capacitor apps 5 days	3b TV support 5 days
4 Ads 2 days	5 Remove Ads 3 days	6 Submission 2 days + 3 to 4 weeks
7 Ops 2 days, ongoing		

 Paperwork and accounts	you · 1 hour of forms · 1 to 2 weeks wait
• D-U-N-S number for RAL (free, 5 to 10 business days). Needed for Organization accounts. • Apple Developer Program, \$99/yr, then Paid Apps Agreement, bank and W-8BEN tax form. Bahrain bank acceptance is unverified; a bank in another country is accepted if needed. • Google Play Console, \$25 once, plus a Play merchant account. Bahrain is officially supported for both. Payout in USD by wire. • AdMob account (Bahrain supported, \$100 payout threshold). Apps get approved only once live in a store. • RevenueCat account (free under \$2,500/month). • Google Cloud consent screen: app name Letterlock, logo, homepage, privacy URL, authorized domain, Search Console verification. • Pick the store name (D1), decide the VPS (D8), give me SSH access.	



Store-readiness changes inside the web app

5 dev days

Everything here ships to the website too, so it pays off before the apps exist.

CHANGE	WHERE	WHY
Account deletion	New <code>delete_my_account()</code> database function, button in the profile view, web page for the Play listing	Apple 5.1.1(v), Play policy
Privacy policy + terms pages	Static pages on <code>letterlock.raltech.dev</code> , linked from Settings and both store listings	Both stores, AdMob, Google brand verification
Sign in with Apple	Apple provider in our auth server (Services ID, Team ID, .p8 key; secret renews every 6 months)	Apple 4.8
<code>/join/CODE</code> links	Router, QR code, copy link; <code>?room=</code> stays as alias	Universal links
Offline handling	Non-blocking banner, local play works offline, remote-media packs hidden offline, rooms disabled, score posts queued	Apple 2.1: reviewers test in airplane mode and IPv6-only
Version gate	<code>/api/app-config</code> with min version + maintenance flag	Retire old app versions, show maintenance notices
Icons + splash	1024 PNG master, generated sets, PWA icons	Stores + PWA
Bundle media	Flags, logo SVGs, fonts served locally	Offline, review reliability
Content changes	D3, D9, D10, D11 applied	Copyright tab
<code>app-ads.txt</code>	Root of the domain listed in the stores	AdMob requires it since Jan 2025



Mobile experience rehaul, portrait and landscape

6 dev days · frontend-stack

Playing on a phone is not great today, and in the browser the URL bar eats the screen in both orientations. This is a full optimization and rehaul of the mobile experience, run through the frontend-stack pipeline before any store screenshot is taken. Details in the "Mobile & TV" section.

- URL bar: the native app has none; the web version gets an "Add to Home Screen" prompt (full-screen PWA) and a Fullscreen button on Android Chrome; layout never shifts when the bar collapses.
- Landscape-first host layout: board left at max height, question and host pad right; slim header; thin timer bar. The phone controller locks to portrait.
- Touch targets at least 44 pt, primary actions in the thumb zone, destructive actions out of the way, type scale re-tuned per breakpoint.
- Keep the screen awake during a match; audio and motion tuned for low-end Android.
- Verified with the 17-device no-scroll matrix, before/after screenshots per orientation, and a real iPhone plus a mid-range Android.

Will this continue for our app? Yes. The apps and the website all talk to `api.letterlock.raltech.dev`. Same database, same accounts, same leaderboards, same saved games, same ads-removed flag. Nothing in the app depends on Supabase cloud. Full explanation in the "Off Supabase" section.

- Docker stack on the VPS: Postgres 17, API gateway, auth server, REST layer, realtime server, functions runtime, admin UI. Remove file storage, image proxy, analytics and log shipper to save about 1.5 GB RAM.
- nginx in front with TLS on `api.letterlock.raltech.dev`, WebSocket headers for realtime, admin UI behind basic auth, firewall 22/80/443 only, IPv6 record.
- Auth config: site URL, redirect allowlist, Google callback on our domain (added in Google Cloud too), Apple provider, Resend SMTP for auth mails.
- Copy the `send-otp` function into the functions volume.
- Data: three dumps (roles, schema, data) restored into the VPS. Password hashes and Google identities survive. **Users sign in again once.**
- Test the host + phone lobby on the VPS first: a known self-host realtime bug can crash broadcast on the default tenant.
- Rewrite CI: build, rsync, migrations by connection string, functions by scp. Point the QA cleanup workflow at the new admin URL.
- Cutover on a quiet evening, keep the cloud project two weeks as rollback, then delete it. From then on there is no Supabase account.

- Capacitor 8, bundle id `dev.raltech.letterlock`, web assets bundled inside the app (never loading the live URL).
- Plugins: app (deep links, Android back), browser, haptics, share, network, status bar, splash, keyboard, preferences, screen orientation.
- Native Google and Apple sign-in via ID tokens. Google blocks OAuth inside embedded web views, and the native sheet shows no domain text at all.
- Deep links: association files on our domain so QR codes open the app if installed, the website otherwise.
- Mirror saves and settings into native preferences (iOS can evict web storage). Handle inline video, first-tap audio unlock, no pinch zoom, Android 16 edge-to-edge.
- iOS builds on Codemagic (500 free macOS minutes a month) straight to TestFlight; Android bundles to the Play internal track. No Mac needed.



TV support: Android TV / Google TV with the remote

5 dev days

- Same Android project, TV-enabled: leanback launcher entry, touchscreen not required, 320×180 TV banner, 1080p TV screenshots, Android TV track review (D-pad navigable, focus always visible, no touch-only paths).
- Remote control: arrows move focus to the nearest control, Enter activates, Back closes the top modal. Arrow keys already move a cursor across the hex board.
- Text and numbers: focusing a field on Android TV opens the system on-screen keyboard; we keep TV inputs minimal (room code, username) and add a big remote-friendly number pad for room codes.
- 10-foot UI: big type, 5% overscan margin, high contrast, nothing reachable only by hover or scroll. No banner ads on TV.
- Works as the shared board with phones as controllers, or alone with the remote in host-adjudicated mode.
- Tested on the Android TV emulator and with a permanent keyboard-only Playwright run at 1920×1080.



Ads

2 dev days + AdMob approval 1 to 2 weeks after launch

- AdMob via the Capacitor community plugin. AdMob only, no mediation until real volume.
- Consent form on every launch before ads (EU/UK/US messages), a "Privacy options" entry in Settings, and the iOS tracking prompt with a short explainer. Declining is fine: ads stay, just untargeted.
- Placement: interstitial on the result screen and between games only, capped; rewarded ad for an extra skip or hint; banner on Home, Category menu, Settings, Lobby. Never on the board, never on the phone controller during a question.



Remove Ads purchase

3 dev days

- One non-consumable product in both stores, \$3.99 tier, Family Sharing on for iOS.
- RevenueCat SDK, entitlement `no_ads`, linked to our user id at sign-in, restore behavior "transfer".
- Restore Purchases button (Apple requires it). RevenueCat webhook writes `profiles.ads_removed` in our database; refunds revoke it automatically.



Store submission

2 dev days + 2 to 4 weeks waiting

- Assets from the real build: iPhone 6.9" and iPad 13" screenshots, Play phone and tablet screenshots, 1024×500 feature graphic. No logos or franchise names in screenshots.
- Forms: App Privacy labels, the new age-rating questionnaire, Play Data safety, content rating, "Contains ads", target audience 13+.
- Reviewer notes with a demo room flow and test account. Budget one rejection round.



Keep it alive

2 dev days, then ongoing

Backups first

Nightly database dump + volumes to offsite storage, 30-day retention, quarterly restore test. Nobody backs up our VPS but us.

Uptime

Uptime Kuma probing the site, auth, REST, realtime and the app-config endpoint, alerting your phone.

Crashes and errors

Sentry free tier plus the stores' own vitals. Play hides apps above a 1.09% crash rate.

Network in the app

Yes, we need it: a connectivity listener, an offline banner, queued writes, and a maintenance flag during VPS work.

Updates

Pinned Docker image tags, manual upgrades, log rotation, fail2ban. Yearly Play API and Xcode bumps. Apple sign-in secret every 6 months.

Takedowns

A published legal contact and a 48-hour removal routine for any logo or fandom complaint.

Mobile and TV: first-class targets

Two additions to the plan: a full rehaul of the phone experience in both orientations, and a TV build you control with the remote.

Mobile: the problems and the fixes

PROBLEM	FIX
Browser URL bar steals space, portrait and landscape	Three layers, all shipped. 1. The native app has no browser chrome at all (the real fix). 2. PWA standalone: an in-app "Add to Home Screen" prompt (iOS Safari steps, Android install prompt) so the web version opens full-screen with no bar, today. 3. In the browser: a Fullscreen button on the board screen for Android Chrome (iPhone Safari does not allow it, so iPhone gets the PWA route), and a layout that never shifts when the bar collapses or expands.
Landscape is cramped	Landscape-first host layout: board left at max height, question and host pad right as one column, header collapses to a slim strip, timer becomes a thin top bar. The phone controller locks to portrait.
Touch targets and thumb reach	Every tappable at least 44 pt; host pad and the controller answer button anchored to the bottom thumb zone; exit and undo moved away from the primary thumb path.
Small text at arm's length	Type scale re-tuned per breakpoint; question text at least 18 px on phones; hex letters scale with the board.
Screen dims or locks mid-match	Keep-awake during a match (native plugin in the app, Wake Lock in Chrome).
Low-end Android jank	Motion budget: claim animation and winning trace stay, ambient particles reduced on data-saver and low-memory devices; measured with CPU throttling.
How we verify	The 17-device no-scroll matrix (including landscape phones) stays the gate, plus before/after screenshots per screen per orientation, plus a real iPhone and a mid-range Android.

Process: this phase runs through the frontend-stack pipeline: style pick, taste floor, guidelines review, Playwright verification. It happens before any store screenshot is taken (decision D16).

TV: which TVs have the Play Store

TV	PLAY STORE?	WHAT THAT MEANS FOR US
Philips (TP Vision models sold in Europe, Middle East, Asia)	Yes	Android TV / Google TV. Our Android build runs, downloaded from the Play Store on the TV.
Sony, TCL, Hisense (most), Sharp, Xiaomi, Nvidia Shield, Chromecast with Google TV, onn	Yes	Same.
LG	No	webOS with the LG Content Store. It accepts HTML5 apps, so the same web build can be packaged later.
Samsung	No	Tizen with Samsung TV Apps. Also accepts HTML5 apps; same later path.
Roku TVs	No	Not planned.
Apple TV	No	No web view, no Capacitor. AirPlay mirroring from an iPhone or iPad already shows the board.
Amazon Fire TV	Own store	Android-based; the Android TV build can go to the Amazon Appstore later.

TV: how it will work with the remote

AREA	WHAT WE DO
Store listing	Same Android project, TV-enabled: leanback launcher entry, touchscreen not required so one app serves phones, tablets and TVs, 320×180 TV banner, 1080p TV screenshots, Android TV track with its own review (D-pad navigable, focus always visible, no touch-only paths, no portrait, Back behaves).
Remote (D-pad)	Remote presses arrive as keyboard events. A spatial focus manager moves focus to the nearest control in the arrow's direction, Enter activates, Back closes the top-most modal or asks to exit. The hex board already supports arrow-key play. One big, high-contrast focus ring everywhere.
Typing text and numbers	Focusing a text field on Android TV pops the system on-screen keyboard automatically; Enter submits, Back dismisses. TV inputs stay minimal (room code, username), plus a big remote-friendly number pad for room codes. Nothing depends on hover.
10-foot UI	Body text at least 24 sp, headings at least 48 sp, 5% overscan margin, blue/amber palette (already TV-safe), no small chips, no scroll-only lists. No banner ads on TV; interstitials still only between games.
Roles on a TV	TV as the shared board while phones are controllers (best), or TV alone with the remote in host-adjudicated mode. Both must work with the remote only.
Testing	Android TV emulator for the real remote path, plus a permanent keyboard-only Playwright run at 1920×1080: every screen completable with arrows, Enter and Escape.

Off Supabase: what that means exactly

You said "we will migrate off of Supabase". Here is precisely what leaves, what stays, and the one choice inside it.

	TODAY	AFTER THE MOVE
Where the data lives	Supabase's servers (their cloud project)	Our VPS, plain Postgres in Docker
Supabase account / bill	Yes	None. Project deleted after cutover.
Domain users see	1kudntyvngwvlzuciocd.supabase.co	api.letterlock.raltech.dev
Who can switch it off	Supabase	Only us
Server software	Supabase's hosted stack	Option A: the same open-source stack, run by us. Option B: our own custom code.
App code changes		Option A: the URL and key. Option B: rewrite of auth, ~30 data calls, realtime sync, leaderboards.

Decision D14: Option A or Option B

A • Run the open-source stack ourselves (recommended)

Supabase publishes its server as free open-source software (Postgres + an auth server + a REST layer + a realtime server). We run those containers on our VPS. No account, no bill, no supabase.co, our domain, our disks. ~3 days. The app keeps working unchanged. This is what the plan assumes.

B • Fully custom backend

Write our own API (Node), own login system (email codes, Google, Apple), own websocket server for host + phone rooms, port ~30 database functions. Same end result for users, 4 to 6 extra weeks, more code to maintain. Possible later at any time because the database is plain Postgres in both options.

Either way, "Supabase" as a company disappears from the picture. The only question is whether we reuse their free server software (A) or write our own (B). Wherever this page or the repo doc says "the backend on the VPS", it means Option A unless you pick B.

Fixing the "xxxx.supabase.co will access" text

Why it happens: Google shows the domain of the login callback until the app's branding is verified. Today that callback is on the Supabase project domain.

- 1** Move the callback to `api.letterlock.raltech.dev` . Automatic with Phase 2.
- 2** Verify `raltech.dev` in Google Search Console.
- 3** Google Cloud branding: app name Letterlock, logo, homepage, privacy and terms URLs on that same domain, authorized domain.
- 4** Set the audience to External and publish. Testing mode hides branding.
- 5** Click Verify Branding. Minutes to three business days. Result: "Sign in to continue to Letterlock" with our logo. In the mobile apps the native Google sheet shows no domain at all.

How do we know who bought Remove Ads?

Short answer: the store account (Apple ID on iPhone, Google account on Android) owns the purchase. Our login does not, unless we copy it there ourselves. We do both.

- 1 Someone taps Remove Ads on an iPhone. Apple charges their Apple ID and records "this Apple ID owns remove_ads". Our account is invisible to Apple and Google.
- 2 Same Apple ID, new phone or reinstall: Restore Purchases re-grants it with no login of ours. Apple requires that button. Android re-grants automatically.
- 3 An iPhone purchase does not appear on Android, and neither appears on the website. Only our server can bridge them.
- 4 Receipts can be faked on a jailbroken phone, so the flag is written only after server-side verification. RevenueCat verifies with Apple or Google and calls our webhook, which sets `ads_removed = true` on the account in our database.
- 5 iPhone, Android and web all read that one flag when signed in. **Ads are hidden if either the store says this device owns it, or the signed-in account has the flag.**

The rule set

SITUATION	WHAT HAPPENS
Buys as a guest, signs in later	The anonymous purchase merges into the account at sign-in; flag set.
Second device, different Apple ID or Android	Store restore fails, but the account flag is true, so no ads. This is the whole point of the flag.
Two of our accounts share one Apple ID	The entitlement transfers to whoever restores last. One account at a time, so one purchase cannot unlock unlimited accounts.
Refund or chargeback	Apple or Google notify RevenueCat, the webhook sets the flag back to false.
Family Sharing (iOS only)	Family devices get it through the store; we do not write the flag to their accounts.
User deletes their account	Flag is gone. Restore Purchases re-creates it on that store account.
Buys on web (later)	Server sets the flag directly; apps honor it when signed in.

What we chose

CHOICE	PICKED	REJECTED
Billing	Apple In-App Purchase + Google Play Billing (mandatory for digital goods)	Stripe inside the app (instant rejection)
SDK	RevenueCat: free under \$2.5k/mo, does receipt checks, refunds, restore, cross-platform, webhooks	Free plugins where we host receipt verification, Apple notifications, Google Pub/Sub and reconciliation ourselves
Type	One-time, about \$3.99	Subscription (wrong vibe for a family game)
Web	Flag honored; optional web purchase later via a merchant of record	Advertising web prices inside the iOS app (Apple 3.1.3 outside the US)

Ads: what to know

- **Placement rules are law, not taste.** Play forbids interstitials at app open, before a level, mid-action, or on exit. End of a game is the allowed natural break. Rewarded ads are always fine because the user opts in.
- **Consent is mandatory** in the EU and UK through Google's own consent SDK, and recommended everywhere.
- **AdMob approval happens after launch.** They check `app-ads.txt` on our domain and the live listing.
- **Money reality.** Revenue per 1,000 ad views: interstitial about \$10 in the US, \$5 in the EU, \$1 to \$4 in MENA; rewarded \$16 to \$20 in the US; banners about \$1. A party game shows two or three interstitials per half-hour session for a whole room on one device.

\$10 to \$40 PER MONTH AT 300 DAILY PLAYERS	\$100 to \$400 PER MONTH AT 3,000 DAILY PLAYERS	\$3.99 REMOVE ADS, LIKELY THE BIGGER LINE EARLY
---	---	---

Where ads go

FORMAT	WHERE	NEVER
Interstitial	Result screen, between games in a best-of series, max 1 per 3 to 5 minutes	Mid-question, during the timer, at app open, on the host pad, on the phone controller while a question is live
Rewarded (opt-in)	Extra skip, a hint, unlock a category for one match	Anything that blocks the turn
Banner	Home, Category menu, Settings, Lobby waiting	The board screen (shared TV screen)
Web	None at launch	

Copyright verdicts

Ads and a purchase make the app commercial, which changes what we can use.

CONTENT	LEGAL	STORE	VERDICT
Guess the Song (1,624 iTunes previews)	High	High	Remove from store build. Apple 5.2.5: previews "may not be used for their entertainment value, e.g. the soundtrack to a game". The API terms require showing the track name and a buy link, which would spoil the answer.
TV Show and Sitcom clips (296)	High	High	Remove. TV previews are not even listed as permitted promo content. Convert to text trivia.
Melody, iTunes part (~212)	High	High	Remove.
Melody, our 23 synthesized public-domain tunes	Low	Low	Keep, bundle, grow to 200+: classical, folk, anthems, nursery, pre-1930 songs.
Logos (~600)	Medium	Medium	Keep with disclaimer, bundle the SVGs, drop the Apple logo, no logos in marketing, removal contact.
Flags	Low	Low	Bundle everything; Sports pack still hotlinks a flag CDN, switch to local.
Charades images (random Flickr)	High	Medium	Drop images, word only. Unmoderated photos in a family game plus licenses we cannot honor.
Fandom packs (17, text only)	Medium	Low	Keep with "Unofficial fan trivia" label, rename Harry Potter and Pokémon packs, franchise names never in store metadata, 48-hour takedown routine.
Movies, TV, Music text trivia	Low	Low	Keep.

Why there is no cheaper music path. Spotify removed preview URLs for new apps in 2024. Deezer's API is non-commercial only and stopped issuing tokens. Apple's MusicKit needs every player to hold an Apple Music subscription and forbids indirect monetization. Every surviving music-quiz app either licenses clips (SongPop) or gates behind Apple Music. Licensed clip vendors exist if music packs ever justify the cost.

"Will Apple ban it for looking AI-made?"

No. Neither store has any rule about who or what wrote the code, and reviewers never see the source. They judge the product. Here is what actually gets apps rejected, and our answer to each.

REAL REJECTION REASON	OUR ANSWER
4.2 "Repackaged website"	Bundle the build inside the app, native haptics, share sheet, deep links, native login, offline mode, in-app purchase.
4.3 Spam or clone (tightened June 2026)	Original mechanic, 40k questions, differentiated name, original icon, honest screenshots.
2.1 Crashes, blank screens, placeholders, IPv6-only failure	Offline banner, hostnames only, no test ads, every URL live.
2.3 Metadata mismatch	Screenshots from the real build; dev seams already inert in production.
5.1.1 No privacy policy or account deletion	Phase 1 adds both.
4.8 Google login without an equivalent private option	Sign in with Apple.
3.1.1 Digital goods outside in-app purchase	Native purchases only, no web pricing mentioned in-app.
5.2 Intellectual property	Copyright tab changes.
Play: personal account skipped the closed test	Organization account, or start the 14-day test early.

Native-feel checklist before submitting

- Splash and icon, styled status bar, no browser chrome, no pinch zoom, no text selection on game surfaces, no link previews.
- Android back button always does something sensible: close modal, exit confirm, go home.
- Safe areas on notch phones and Android 16 edge-to-edge; landscape phones verified with the existing no-scroll checker.
- Haptics through the native plugin, native share sheet, in-app purchase, offline path.
- Privacy policy, support page, in-app account deletion, Sign in with Apple.
- Honest metadata, screenshots from the real build, age rating, privacy labels, data safety form, 13+ audience.
- Reviewer notes: demo room code, test account, how to play host and player.

Costs, timeline, risks

Costs

ITEM	COST
Apple Developer Program	\$99 per year
Google Play Console	\$25 once
D-U-N-S number	Free
AdMob, RevenueCat under \$2.5k/mo, Sentry, Uptime Kuma, Codemagic 500 min/mo, the open-source backend stack	Free
VPS with 8 GB RAM	What you pay today, or about \$20 to \$40/mo if upgraded
Offsite backups	\$1 to \$2 per month
Store cut	15% of purchases
Optional later	Capgo OTA \$12/mo, licensed music clips (quote), Paddle or Lemon Squeezy about 5% + \$0.50 per web sale

Timeline

PHASE	DEV DAYS	WALL-CLOCK	DEPENDS ON
0 Paperwork	0.5 + your forms	1 to 2 weeks	You
1 Store-ready web app	5	Weeks 1 to 2	Nothing, can start now
1b Mobile experience rehaul	6	Weeks 2 to 3	Phase 1
2 Backend to VPS	3 + cutover	Weeks 1 to 3	VPS access
3 Capacitor apps	5	Weeks 4 to 5	Phases 1 and 1b, Apple account
3b TV support (Android TV, remote)	5	Weeks 5 to 6	Phase 3
4 Ads	2	Week 6	Phase 3, AdMob account
5 Remove Ads	3	Weeks 6 to 7	Phase 3, store products
6 Submission	2 + waiting	Weeks 7 to 10	Everything above, TV track review
7 Ops	2	Week 2 onward	Phase 2
Total	~33	8 to 10 weeks to live	

Risks and unverified items

- Apple accepting a Bahraini bank for payouts is unverified. Fallback: a bank in another country or the company's bank.
- A self-hosted realtime bug can crash broadcast on the default tenant. We test the host + phone lobby before any cutover.
- Exact "Letterlock" name is taken. Reserve the chosen name as soon as the Apple account exists.
- Apple universal links with query strings are unreliable, hence `/join/CODE`.
- Anti-steering rules in the US and EU keep changing. We never mention web purchases in-app, so changes cannot hurt us.
- MENA ad rates are low. Remove Ads is the revenue line.
- No managed backups on the VPS. Backups are the first ops task, before cutover.

Canonical version with all sources: `LAUNCH_PLAN.md` in the repo. No code changed yet; decisions D1 to D16 are pending.